

REST API	Finding	Class	Severity	Contract Evidence	Actual	Expected	Why It Matters?	Recommendation	Acceptance Criteria	Blocking?
POST /accounts	Account request properties are not required.	Validation	P1	"handler.createAccountRequest" defines "document_number", but the schema has no "required" list.	Request body is required, but its "document_number" is optional.	Contract should identify which fields are mandatory for a valid create request.	Request such as {} can be schema-valid even though the operation have no meaningful interpretation.	Add "required" properties to the request schema. required: - document_number	1. Schema validation rejects an account creation request when "document_number" is missing. 2. Negative tests have a deterministic expected status and error response.	Yes
POST /accounts	"document_number" has no validation rule	Validation	P1	"document_number" is only declared as a string with an example. There is no "minLength", "maxLength", "pattern", or other format constraint.	Any string is schema-valid, including an empty string or a string containing non-digit characters.	Contract should define the valid document-number shape.	Backend engineer says "Document numbers are validated — something like ten to fourteen digits", but that is not contractual.	Add both the length and numeric-only constraint after confirming with the PO/dev. document_number: example: "12345678900" type: string minLength: 10 maxLength: 14 pattern: "^[0-9]{10,14}\$"	1. Exact valid/invalid boundaries are documented. 2. Boundary tests exist for the first invalid length, valid range, and last invalid length. 3. Character rules are defined.	Yes
POST /accounts	Duplicate "document_number" behavior is undefined	Contract Ambiguity	P1	"POST /accounts" documents only 201, 400, and 405. No uniqueness rule is defined.	Contract does not say whether duplicate documents are allowed, rejected, or mapped to an existing account.	If "document_number" identifies a unique cardholder account, duplicate creation should have an explicit outcome.	Ambiguous duplicate handling can create multiple accounts with one document_number.	Confirm whether "document_number" is unique and document the resulting status/error. Example: API returns 409 Conflict when "document_number" already exists.	1. Two creates with the same document number have a defined outcome. 2. The expected HTTP status is documented. 3. The response body is defined.	Yes
POST /accounts	Same error message "account not found" for 4xx errors	Validation	P1	"POST /accounts" documents 401 and 405 with error message "account not found".	"POST /accounts" documents 401 and 405 with error message "account not found".	The condition and status should have a deterministic mapping.	Proper error codes/messages removes ambiguity for consumers.	Document the exact error message for 4xx errors. Example: 1. API returns 400 Bad Request when "document_number" is invalid. 2. API returns no body for 405 Method not allowed.	Negative tests have a deterministic expected status and error response.	Yes
GET /accounts/{accountId}	Same error message "account not found" for 4xx errors	Validation	P1	"POST /accounts" documents 400, 404 and 405 with error message "account not found".	"POST /accounts" documents 400, 404 and 405 with error message "account not found".	The condition and status should have a deterministic mapping.	Proper error codes/messages removes ambiguity for consumers.	Document the exact error message for 4xx errors. Example: 1. API returns 400 Bad Request when "accountId" is invalid. 2. API returns 404 Not Found when "accountId" doesn't exist. 3. API returns no body for 405 Method not allowed.	Negative tests have a deterministic expected status and error response.	Yes
POST /transactions	Transaction request properties are not required.	Validation	P1	"handler.createTransactionRequest" defines "account_id", "amount", and "operation_type_id", but the schema has no "required" list.	Request body is required, but its individual properties are optional.	Contract should identify which fields are mandatory for a valid create request.	Request such as {} can be schema-valid even though the operation have no meaningful interpretation.	Add "required" properties to the request schema. required: - account_id - amount - operation_type_id	1. Schema validation rejects a transaction request missing any of the three fields. 2. Negative tests have a deterministic expected status and error response.	Yes
POST /transactions	Non existent account behavior is not defined	Contract Ambiguity	P1	Contract documents 400 and 422 but does not attach either status to a specific business condition. The "errorResponse" example says "account not found".	The example suggests an "account not found error" exists, but the contract does not say whether it is 400 or 422.	The condition and status should have a deterministic mapping.	Proper error codes/messages removes ambiguity for consumers.	Document the exact status for an unknown account and the error shape. Example: API returns 404 Not Found with error message "account not found".	A transaction against a non-existent account always returns the documented status and error code/message shape.	Yes
POST /transactions	"amount" has no decimal precision, range, or sign constraint.	Money Safety	P0	The request defines "amount" as only "number". The endpoint description says the caller sends a positive amount, but the schema itself does not enforce decimal precision, positivity, maximum value, or minimum value.	Zero, negative, and extremely large values are all schema-valid.	Contract should define the amount range and decimal precision to test it.	This is the area where money-related bugs are most likely to happen. If we don't clearly define the allowed values and decimal precision, it's difficult to know what the correct expected result should be.	Define: - allowed decimal precision - minimum - maximum - behavior for zero and negative values.	For each boundary, the contract specifies the expected outcome and status: - zero - negative - normal positive - maximum supported amount - above maximum - maximum supported decimal precision - above supported precision	Yes
POST /transactions	Currency and API amount unit are unspecified.	Money Safety	P0	The request defines "amount" as only "number". There is no currency field or unit declaration anywhere in the supplied contract.	Backend engineer says "Amounts go in as rupees, we store them in the smallest unit internally", but neither point appears in the contract.	API should make the amount representation unambiguous.	A value such as 50 is not sufficient to establish whether the caller means ₹50, 50 paise.	Document the currency representation explicitly. Example: Add required parameter "currency" to the request body.	A caller can determine from the contract exactly what 50, 50.5, and the supported amount mean.	Yes
POST /transactions	"operation_type_id" accepts any integer	Money Safety	P0	"operation_type_id" is declared only as an integer. The transaction description lists four operation IDs, but contract don't define any constraint.	Schema allows any integer.	Only supported operation types should be valid.	An unknown operation type can reach business logic without the contract defining what it means.	Add "enum" constraint for "operation_type_id". enum: - 1 - 2 - 3 - 4	1. IDs 1, 2, 3, and 4 are valid. 2. An unknown ID is invalid. 3. The invalid status and error body are documented.	Yes

POST /transactions	Idempotency is absent from the contract	Money Safety	P0	No idempotency header is documented.	Backend engineer says "There's an idempotency header on transactions so retries don't double-charge. Send the same key, you get the original response back.", but is not represented in the contract.	Idempotency header must be documented.	Idempotency is critical to preventing double charge during retries.	Define: - header name - whether it is required - key format/length - response/status for replay	Same key + same body → No double charge.	Yes
POST /transactions	Idempotency key reuse with a different body is undefined	Money Safety	P0	No idempotency header is documented.	No behavior is specified.	An idempotency key must not become associated with multiple financial operations.	A client can accidentally or maliciously reuse a key for a different transaction payload. The provider needs a deterministic rule.	Define a conflict response for a different payload.	1. Same key + different body → documented conflict. 2. Missing key → documented behavior.	Yes
POST /transactions	Operation-specific mapping is not contractual	Money Safety	P0	The contract says "Operation type IDs: 1=Normal Purchase, 2=Purchase with installments, 3=Withdrawal, 4=Credit Voucher. Send a positive amount. The server applies the correct sign based on operation type.". The response example contains "amount: -100.5" and "type: debit".	"amount" sign is not defined for each ID. The contract gives one negative example but does not define the complete mapping. Backend engineer says "Purchases and withdrawals go negative, credit vouchers go positive."	The mapping between each operation and its financial sign/type should be explicit.	An incorrect sign can lead to money integrity failure.	Document a mapping for all operation IDs, including expected response "type" and "amount" sign.	For each of IDs 1 to 4: - resulting sign is defined - resulting transaction type is defined - response assertion can be generated from the contract.	Yes
POST /transactions	"event_date" is a string without "date-time" format	Validation	P2	"event_date" is declared as "string" with an ISO-looking example.	Any string is schema-valid.	Event timestamp format should be contractual.	The consumer should clearly understand the format of the timestamp.	Add "format: date-time" to the contract.	Schema validation rejects non-date-time strings and accepts the documented timestamp representation.	No
POST /transactions	Response "type" is an unrestricted string	Money Safety / Validation	P1	The response defines "type" as string, example "debit".	Any string is schema-valid.	Transaction types should be constrained if consumers depend on it.	A consumer could receive an unexpected transaction type without violating the schema.	Define an enum or a documented mapping between operation and response type.	Unknown transaction types fail schema validation and the valid set is documented.	Yes
POST /transactions	Audit-write guarantee is not represented	Money Safety / Observability	P0	Contract contains no response field, or header representing audit creation.	The audit-row statement exists only as Backend engineer hearsay.	If every write must produce an audit record, there needs to be a defined contract or a non-API verification mechanism.	A financial transaction without a corresponding audit trail can become difficult or impossible to reconcile.	Define the audit guarantee and provide a verification mechanism via API, or queryable audit store in a test environment.	A test that can prove that a successful write has a corresponding audit record.	Yes
POST /transactions	Audit/business-write atomicity cannot be verified	Money Safety	P0	Contract doesn't define how database transactions are handled.	Backend engineer says "Every write also writes an audit row, in the same database transaction, so they can't diverge", but this is not present in contract.	Provider should either clearly guarantee that the operation happens completely or not at all in documentation, or provide a way for us to verify this through integration testing.	This is critical but currently cannot be tested from the published API.	Document the guarantee and define how it can be verified.	A forced failure between business-write and audit-write results in neither being committed.	Yes
POST /accounts POST /transactions	Request ID response header is not contractual	Observability	P1	Contract defines response bodies but no response headers.	No request ID header is documented.	Request ID header should be defined.	The stated tracing guarantee across services cannot be validated from the contract.	Define the header name, type, and format.	Every response includes the tracing header, including error responses.	No
POST /accounts POST /transactions	Response properties are not required	Validation	P1	Neither "accountResponse" nor "transactionResponse" declares a "required" list.	The schema allows a successful response with any or all properties absent.	Consumers should know which fields are guaranteed in a successful response.	For a transaction response, missing "transaction_id", "amount", "account_id", or operation information can make the result unusable for reconciliation.	Declare required response properties.	A response missing a mandatory field fails schema validation.	No
POST /accounts POST /transactions	Extra request properties are not constrained	Validation	P2	The request objects do not declare "additionalProperties: false".	The contract is silent.	The provider should document whether unknown fields are ignored or rejected.	Contract should defined if additional properties are rejected. If not, provider should ignore them.	Choose and document an explicit policy.	The behavior of an unknown request field is deterministic and documented.	No
POST /accounts GET /accounts/{accountId} POST /transactions	Authentication/Authorization contract requirement is absent	Security	P0	Contract does not define an authentication or authorization requirement for the Transactions Service.	No authentication/authorization requirement is documented.	Authentication and authorization requirements defined in the contract.	Consumers cannot determine or test who may mutate financial state.	Contract should define: - authentication mechanism - required credentials/header/token - authorization expectations - expected response for unauthenticated requests - expected response for authenticated but unauthorized requests	Acceptance criteria: 1. authentication mechanism is explicitly defined in the contract for each endpoint. 2. Authentication and authorization failure statuses are documented. 3. Security-related headers/credentials are defined. 4. Contract/API test verifies that unauthenticated access cannot perform operations. 5. Authorization test verifies that an authenticated caller without the required permission cannot perform the operation.	Yes

Backend Engineer Hearsay	Contract Position	Classification								
Amount is in rupees	No currency is defined	Silent								
Amount stored internally in smallest unit	Internal storage is not exposed	Silent								
Caller sends positive amount	Explicitly stated	Supported								
Purchase/withdrawal negative, voucher positive	"Correct sign" is stated, but full mapping is not defined	Incomplete								
Idempotency header exists	No header documented	Silent								
Same key returns original response	No replay behavior documented	Silent								
Document numbers are 10–14 digits	No constraints	Silent								
Every response has request ID	No response header documented	Silent								

Every write creates audit row	Not represented	Silent							
Audit and business write are one DB transaction	Internal atomicity not represented	Silent							